applied sciences

MDPI

Article

Question–Answer Models for Teaching Programming in Kazakh: A Morphology-Aware Controlled Study

Ualsher Tukeyev [1], Bekarys Rysbek [1,*] and Bogumiła Hnatkowska [2]

[1] Faculty of Information Technology and Artificial Intelligence, Al-Farabi Kazakh National University, Almaty 050040, Kazakhstan; ualsher.tukeyev@kaznu.edu.kz
[2] Faculty of Computer Science and Management, Wroclaw University of Science and Technology, 50-370 Wrocław, Poland; bogumila.hnatkowska@pwr.edu.pl
* Correspondence: rysbek_bekarys@kaznu.edu.kz

Abstract

This study investigates whether morphology-aware input processing can improve question-answering performance for teaching programming in Kazakh, a low-resource language with rich morphological variation. Rather than proposing a new neural architecture, the study presents a controlled empirical evaluation of an adapted pipeline that combines FEM-Seg_kaz preprocessing with morphology-aware post-processing (MAPP) normalization and domain-specific fine-tuning. A dataset of 50,386 Kazakh-language Python programming question–answer pairs was constructed with GPT-4o assistance and expert validation. Each pair consists of a student-style programming question in Kazakh and a corresponding instructional answer, usually including a short explanation and, when appropriate, a Python code example. Experiments with multilingual encoders and a controlled MiniLM setup show that morphology-aware normalization produces measurable improvements under fixed evaluation conditions, while its combination with domain-specific fine-tuning improves several answer selection and contextual-overlap indicators. The findings suggest that carefully controlled adaptation of established NLP techniques can support the development of Kazakh educational QA systems in morphologically rich, low-resource settings. Because no separate validation split or early stopping was used, these findings should be interpreted as controlled fixed-epoch comparison results rather than as fully optimized model selection results.

Keywords: morphology-aware QA; fine-tuning; Kazakh; programming education; MiniLM; FEMSeg_kaz; MAPP

Check for updates

Academic Editor: Douglas O'Shaughnessy

Received: 21 May 2026
Revised: 11 June 2026
Accepted: 12 June 2026
Published: 17 June 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Question-answering systems are now widely used in education, healthcare, law, and customer support. These systems help users quickly obtain information and receive answers that match the context of a question [1]. In programming education, they can also support students during learning. For example, they may explain programming concepts, help students understand coding errors, or answer common questions.

This work focuses on programming education in Kazakh. In this area, digital learning materials and specialized QA systems are still limited.

Kazakh NLP resources have grown in recent years, but they remain more limited than those available for high-resource languages. KazQAD introduced an open-domain Kazakh QA benchmark with both reading comprehension and answer selection evaluation settings.

Even so, several challenges still remain. Earlier work in Kazakh and Turkic NLP often notes limited annotated data, morphological complexity, and a lack of educational QA tools tailored to specific tasks.

One important challenge in Kazakh QA is morphology. Kazakh uses many suffixes, case endings, and word-form variations. Because of this, the same meaning may appear in different surface forms. Two students may ask almost the same thing but still look different lexically. This can make answer selection less reliable for multilingual encoder models [2].

To address these challenges, it is important to examine whether specialized linguistic processing and targeted training can reduce the effect of Kazakh's rich morphology on model performance.

In this work, we use FEMSeg_kaz, a morphology-aware segmenter developed for Kazakh and other Turkic languages. Previous benchmarking studies showed that it performs well for these languages.

Unlike standard subword tokenizers, FEMSeg_kaz identifies morpheme boundaries directly. This helps the model better capture the semantic structure of complex word forms.

We investigate this problem through three research questions:

RQ1. Does domain-specific fine-tuning improve answer selection for Kazakh programming questions when multilingual encoders are used?

RQ2. Does morphology-aware input processing improve answer selection quality when the model backbone, data split, candidate answer set, and evaluation protocol are kept the same?

RQ3. Does using FEMSeg_kaz preprocessing, MAPP normalization, and domain-specific fine-tuning together produce better results than using preprocessing or fine-tuning separately?

The study is organized into two distinct experiments. The first focused on RQ1, evaluating the impact of domain-specific fine-tuning on multilingual encoder models for Kazakh programming QA.

The second experiment focused more directly on the linguistic side of the problem and addressed RQ2 and RQ3. First, the experiment tested whether morphology-aware preprocessing improves results on its own, with the model backbone and evaluation setup held constant (RQ2). Second, it examined whether combining linguistic preprocessing with neural fine-tuning produces complementary performance changes across the reported metrics. More specifically, the study examined whether adding domain-specific fine-tuning to the FEMSeg_kaz + MAPP representation changes answer selection quality compared with using preprocessing or fine-tuning separately (RQ3). This experimental design made it easier to separate the effects of neural domain adaptation and morphology-aware preprocessing. The preprocessing stage also builds on earlier studies related to Kazakh morphology-aware segmentation [3].

The paper makes four main contributions, framed as a controlled adaptation and empirical evaluation of established NLP techniques for a specific low-resource educational QA setting.

First, the study provides a controlled morphology-aware QA evaluation framework for Kazakh programming education. The framework combines existing components—FEMSeg_kaz preprocessing, deterministic MAPP normalization, and domain-specific fine-tuning—within a shared answer-selection pipeline. The contribution is not a fundamentally new neural modeling paradigm; rather, it is the controlled isolation of how these components affect Kazakh educational QA when the model backbone, data split, candidate answer set, and evaluation protocol are kept fixed. Leak-resistant pairing across different input representations was also used to make comparisons between configurations more reliable.

Second, the study presents a Kazakh-language QA dataset for Python programming education. The dataset includes 50,386 question–answer pairs. GPT-4o was used at the initial data generation stage. After that, domain-aware experts reviewed, filtered, corrected, and validated the generated data.

Third, the paper compares three multilingual encoder models before and after domain-specific fine-tuning under the same answer selection setting.

Finally, the study examines the relationship between automatic metrics, interval-based human evaluation, and pedagogical usefulness in a low-resource educational QA setting. This contribution offers insights into how automated systems can be reliably evaluated in low-resource educational contexts.

The rest of the paper is organized as follows. Section 2 reviews related work on multilingual QA, domain-specific fine-tuning, educational QA evaluation, Kazakh NLP resources, and morphology-aware processing. Section 3 describes dataset construction, expert cleaning, evaluated encoder models, preprocessing, fine-tuning, and evaluation. Section 4 presents the experimental findings together with the human evaluation results. Section 5 discusses the study's main findings, methodological implications, and limitations. Section 6 concludes the paper and outlines directions for future work.

2. Related Work

Question-answering systems have advanced significantly over time. Earlier QA systems mainly used rule-based methods, lexical matching, and statistical ranking techniques. Modern QA systems rely more on transformer-based neural architectures. These models can capture linguistic context more effectively and better identify semantic relationships between questions and answers [4,5].

Multilingual QA research has also developed in recent years. New benchmarks focus more on native-language data and multilingual evaluation settings. TyDi QA is one example of this direction [6–8].

Fine-tuning is still widely used for low-resource and domain-specific tasks. It helps adapt pretrained models to a specific domain or language. Fine-tuning can improve embedding quality and strengthen the relationship between questions and answers in the embedding space. This becomes especially useful when the target domain differs from the original pretraining data [9–11].

Evaluation across languages is also important in multilingual QA. A system may perform well in one language but much worse in another. Datasets such as MIRACL demonstrate this clearly. They show the importance of evaluating monolingual answer selection in both high-resource and low-resource languages [12,13].

Educational QA systems should not be evaluated solely using automated metrics. Measures such as Exact@1, cosine similarity, and BERTScore can estimate lexical overlap and semantic similarity. However, they cannot fully show whether an answer is correct, complete, or useful for learning. Because of this, human evaluation is still important in educational QA research [14–18].

Earlier studies in Kazakh QA and Turkic NLP provide useful benchmarks and evaluation results. However, they do not directly examine morphology-aware answer selection in Kazakh programming education [19–21].

Another important research area is morphology-aware processing for morphologically rich and low-resource languages. Recent segmentation studies show that morphological segmentation can help when training data are limited. Still, segmentation alone is often not enough. The final results also depend on how the segmented text is normalized and how the model uses the segmented representation [22–24].

FEMSeg_kaz is based on recent CSE-guided studies on Kazakh morphology-aware segmentation and tokenization [25,26]. These studies provide a useful methodological foundation for the current work.

At the same time, they do not fully examine whether morphology-aware preprocessing improves encoder-based answer selection for Kazakh programming QA under controlled experimental conditions.

3. Materials and Methods

3.1. Study Design

The main goal of the experiment was to examine whether morphology-aware input processing can improve encoder-based answer selection for Kazakh programming questions.

The study also investigated whether this improvement remains visible after standard domain-specific fine-tuning.

This problem is important because Kazakh is a morphologically rich language. Similar questions can appear in different surface forms due to suffixation, case marking, and word-form variation. As a result, multilingual encoder models may produce less stable representations of questions.

The study included two experimental stages. The first stage examined the effect of domain-specific fine-tuning on three multilingual encoder models using the same expert-cleaned Kazakh programming QA corpus. The goal here was to see whether fine-tuning improves the overall alignment between programming questions and their corresponding answers.

The second stage focused more on the linguistic side of the study. A controlled MiniLM-based comparison was used to examine whether morphology-aware preprocessing still improves results when the main experimental conditions remain unchanged. In this stage, the model backbone, data split, candidate answer set, inference procedure, and evaluation protocol were kept fixed. This helped make the comparison more controlled. It also made it easier to connect performance differences to the input representation and its interaction with fine-tuning rather than to other factors.

This design made it possible to separate two related effects. One effect comes from neural adaptation through fine-tuning. The other comes from morphology-aware preprocessing. The next subsections explain the preprocessing workflow, input configurations, fine-tuning setup, and evaluation metrics in more detail.

Figure 1 summarizes the overall workflow of the study, from curriculum-guided dataset construction to morphology-aware input configurations, answer selection, and evaluation.

3.2. Dataset Construction and Expert Cleaning

The dataset of Kazakh-language Python programming question–answer pairs was created in two stages. The programming examples were written and checked for Python 3.10 (Python Software Foundation, Wilmington, DE, USA). First, question–answer pairs were generated with LLM assistance. After that, the generated data were reviewed and cleaned by experts.

Before large-scale generation, the topic structure of the dataset was defined using the school programming curriculum and the teacher's long-term instructional plan. The main Python programming topics were selected according to the planned sequence of instruction, including introductory programming concepts, variables, data types, operators, conditional statements, loops, functions, data structures, file operations, error handling, and object-oriented programming. Therefore, the dataset generation process was curriculum-guided rather than topic-free or random.

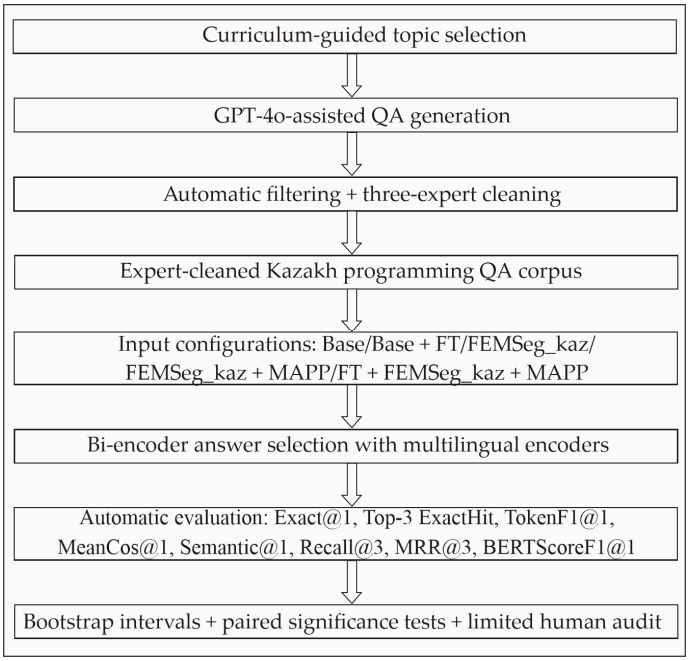


Figure 1. Overall workflow of the morphology-aware Kazakh programming QA evaluation pipeline.

To ensure reproducibility, the prompt template was prepared before large-scale generation began. The model was asked to generate Kazakh-language question–answer pairs for Python programming education:

"Generate Kazakh-language question–answer pairs for teaching Python programming. Each question should be written as a student might ask it. Each answer should be technically correct, clear for beginner learners, and pedagogically useful. If the topic requires code, include a short Python example and explain how it works. Avoid vague, duplicated, or off-topic questions. Structure the output as question–answer pairs."

Each generated item included a student-style programming question and a corresponding instructional answer. If the topic required code, the answer also included a short Python example and a brief explanation. The generated outputs were treated as candidate question–answer pairs rather than final training data. They were subsequently checked through automatic filtering and expert review to ensure technical correctness, topical relevance, linguistic clarity, and pedagogical usefulness.

Although the corpus was not collected directly from classroom dialog or learning management system logs, it was not generated without pedagogical constraints. The content was anchored in a teacher-defined long-term plan and school-level programming curriculum. Therefore, the dataset reflects the intended instructional coverage of Python programming topics rather than arbitrary LLM-generated content. At the same time, curriculum-guided generation is not identical to naturally occurring student interaction. The corpus may still underrepresent informal phrasing, incomplete questions, and spontaneous learner errors that appear in real classrooms.

The dataset was therefore designed as a curriculum-guided and expert-validated educational QA benchmark rather than as a direct collection of naturally occurring student questions. It should not be interpreted as a random GPT-generated corpus. The topic structure was prepared according to the teacher's long-term instructional plan and the school programming curriculum, which helped ensure planned topic coverage and pedagogical alignment. In addition, the three-expert review improved technical correctness, linguistic clarity, code validity, and instructional usefulness. However, curriculum alignment and expert validation do not fully replace authentic learner data. Real student questions may

include incomplete wording, informal phrasing, spelling variation, code-specific context, mixed-language expressions, and learner misconceptions that are difficult to reproduce through LLM-assisted generation alone. Therefore, the reported results should be interpreted as evidence from a controlled curriculum-aligned benchmark, not as direct proof of generalization to spontaneous classroom questions.

Examples of generated question–answer pairs are shown in Table 1. These examples illustrate the basic structure of one dataset item. In this study, a question–answer pair consists of a Kazakh programming question and its corresponding pedagogical answer. The answer provides a direct explanation of the concept and, where appropriate, includes a short code example.

Table 1. Examples of generated Kazakh programming question–answer pairs.

Kazakh Version	English Version
Python тілінде рекурсия қалай жұмыс істейді? Рекурсия—бұл функцияның өзін-өзі шақыруы. Мысалы: def factorial(n): return 1 if n == 0 else n * factorial($n - 1$)	How does recursion work in Python? Recursion is when a function calls itself. For example: def factorial(n): return 1 if n == 0 else n * factorial($n - 1$)
Python тілінде while және for циклдерінің айырмашылығы қандай? While циклі шарт ақиқат болғанша орындалады. For циклі тізім немесе диапазон бойынша қайталанады.	What is the difference between while and for loop in Python? While loop runs as long as a condition is true. For loop runs over a sequence or range.

Note: In Python code snippets, the symbol '*' denotes multiplication and is kept as part of the correct Python syntax.

At the initial stage, 51,264 candidate question–answer pairs were generated using the GPT-4o API with the gpt-4o-2024-08-06 model version. Throughout this paper, the term GPT-4o refers to this model version.

This approach made dataset development much faster. However, it also required a careful validation process after generation. This step was needed to check the pedagogical accuracy of the content and the naturalness of the language. To clarify the methodology, Table 2 summarizes the main trade-offs in this process. It shows both the key advantages and the main limitations of using LLM-assisted data generation.

Table 2. Advantages and limitations of the generated data.

Advantages	Limitations
Speed—Thousands of times faster than manual collection.	Possible Errors—The model may sometimes generate meaningless or incorrect questions.
Volume—Enables the creation of large-scale data.	Human Supervision Required—Experts must review the data to ensure 100% reliability.
Diversity—Can automatically generate questions on a wide range of topics.	Limited Context—Some technical questions may be unclear for the model.

The generated outputs were not used directly as final training data. Instead, they were treated as candidate question–answer pairs. These pairs were later checked through automatic filtering and expert review.

To reduce hallucinations, malformed, or semantically unrelated items, the generated candidates were first automatically checked. This automatic screening used cosine similar-

ity over sentence embeddings to identify potentially weakly aligned question–answer pairs. The embeddings for this preliminary filtering stage were computed using the pretrained sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 model before any task-specific fine-tuning on the present dataset. The threshold of 0.85 was used as a preliminary quality control signal to flag candidate pairs that required further expert inspection. It was not used as an independent final deletion rule. Flagged items were subsequently reviewed by the three experts together with the predefined acceptance criteria, including technical correctness, direct relevance to the question, beginner-level clarity, code validity, and grammatical clarity of the Kazakh text. Therefore, the embedding-based filter supported the expert cleaning process but did not replace human validation. Empty, malformed, off-topic, and low-quality candidates were removed at this stage. Exact duplicates were also removed after normalizing whitespace, punctuation, and letter case.

The filtering pipeline handled both exact duplicates and near-duplicate items.

First, malformed, empty, or off-topic outputs were automatically removed. After that, exact duplicates were removed by normalizing whitespace, punctuation, and letter case.

Next, semantically similar items were checked using sentence-embedding similarity together with manual review.

A pair was treated as a near-duplicate when it expressed the same programming concept and required nearly the same answer, even if the wording was different. For example, two questions were considered near-duplicates if they asked about the same Python operation, concept, or function and led to the same instructional explanation. Differences in wording or word order alone were not considered sufficient to make the items unique. Potential near-duplicates were first identified using sentence-embedding similarity. After that, they were checked manually.

Items were removed only when they repeated the same learning purpose and did not introduce a different concept, difficulty level, example, or programming context.

This step helped remove questions that looked different on the surface but served the same instructional purpose. This was necessary because LLM-generated text often produced multiple paraphrases of the same Python concept. In the final dataset, only useful and non-redundant items were retained.

After automatic filtering, the remaining question–answer pairs were manually reviewed by three domain-aware experts. The expert group consisted of one second-year doctoral student from Al-Farabi Kazakh National University, one second-year master's student from Al-Farabi Kazakh National University, and one second-year master's student from Almaty University of Power Engineering and Telecommunications. The experts had relevant academic experience in programming, Kazakh-language educational content, and computer science education.

The experts reviewed the generated candidate pairs for technical correctness, direct relevance to the question, beginner-level clarity, code validity, and grammatical clarity of the Kazakh text. Items that were unclear, repetitive, weakly aligned with the question, technically incorrect, or pedagogically unhelpful were revised, merged with similar items, or removed. Disagreements and ambiguous cases were resolved through discussion among the three experts, using the predefined acceptance criteria and the curriculum-guided topic structure as the basis for the final decision.

It is important to distinguish this corpus-cleaning stage from the later human evaluation reported in Section 4.3. The corpus-cleaning stage was conducted before model training and evaluation to validate the generated question–answer pairs. The human evaluation stage in Section 4.3 used the same three experts, but its purpose was different: it assessed the correctness of model-selected answers on a 200-item audit sample after model inference.

To keep the dataset reliable and to maintain consistency across the three experts, each entry was checked against five strict acceptance criteria:

1. The answer had to be technically correct.
2. The answer had to match the meaning of the question directly.
3. The explanation had to be clear enough for beginner programmers.
4. Any Python code example had to be syntactically correct and consistent with the explanation.
5. The Kazakh text had to be grammatically clear and understandable.

Minor linguistic, formatting, or code-style inconsistencies were corrected during expert review.

The cleaning stage was important because duplicate, inconsistent, and imperfect entries were common in the raw generation output. Representative examples of duplicated and generated items requiring manual correction are presented in Tables 3 and 4, respectively. Table 3 presents duplicated question–answer pairs, whereas Table 4 presents generated question–answer pairs that required manual correction.

Table 3. Examples of duplicated question–answer pairs.

in the Kazakh Language	in the English Language
Сұрақ: Python-да биттік операциялардың нәтижесін қалай өңдейміз? Жауап: *ф*ункциялар, методтар арқылы.	question: How do we process the results of bitwise operations in Python? answer: By using functions, methods.
Сұрақ:Python-да биттік операциялардың нәтижесін қалай өңдейміз?, Жауап: *ф*ункциялар және методтар арқылы.	question: How do we process the results of bitwise operations in Python?, answer: By using functions and methods.

Table 4. Examples of generated question–answer pairs requiring manual correction.

in the Kazakh Language	in the English Language
Сұрақ: Python тіліндегі @property декораторының мақсаты қандай?, Жауап: “Ол қасиеттерді (properties) анықтау үшін қолданылады.	question: What is the purpose of the @property decorator in Python?, answer: It is used to define properties.
Сұрақ: Python-да проперти дегеніміз не?, Жауап: Бұл атрибуттарды қорғау және олардың мәндерін басқару үшін қолданылатын механизм.	question: What is a property in Python?, answer: It is a mechanism used to protect attributes and manage their values.

Overall, the combined automatic filtering and expert cleaning process reduced the dataset from 51,264 generated candidate pairs to 50,386 validated question–answer pairs. Thus, 878 generated items were removed or merged during the automatic filtering and three-expert cleaning process. In addition, minor linguistic, formatting, and code-style corrections were applied to retained items when necessary; these retained corrections did not reduce the final dataset size. The final 50,386 expert-cleaned pairs were then used for model training and evaluation. The dataset generation and cleaning process is summarized in Table 5.

Table 5. Summary of dataset generation and expert cleaning process.

Stage	Description	Number of Pairs
Initial generation	GPT-4o-generated candidate question–answer pairs	51,264
Automatic filtering and three-expert cleaning	Malformed, off-topic, weakly aligned, duplicate, ambiguous, or technically weak items removed or merged	878
Final expert-cleaned dataset	Validated question–answer pairs used for model training and evaluation	50,386
Minor corrections to retained items	Linguistic, formatting, and code-style corrections applied to retained items	Not counted as removed items

3.3. Evaluated Encoder Models

Three multilingual encoder models were selected for evaluation:

- Sentence-Transformers/paraphrase-multilingual-MiniLM-L12-v2 (https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2, accessed on 21 May 2026)
- Sentence-Transformers/distiluse-base-multilingual-cased-v1 (https://huggingface.co/sentence-transformers/distiluse-base-multilingual-cased-v1, accessed on 21 May 2026)
- bert-base-multilingual-cased, accessed on 21 May 2026 (https://huggingface.co/google-bert/bert-base-multilingual-cased, accessed on 21 May 2026)

These models were selected because they represent different types of multilingual encoders and different computational characteristics. The two Sentence-Transformers models were included because they are designed for sentence-level representation learning and perform well on semantic similarity and answer selection tasks. The first was used in this study as one of the main encoder models for domain-specific QA experiments in Kazakh programming education. The multilingual BERT model was included as a commonly used transformer baseline for cross-lingual representation learning. It supports more than 100 languages and is widely used for cross-lingual representation learning.

3.4. Morphology-Aware Input Preprocessing

The main methodological focus of this paper is a controlled MiniLM-based comparison of morphology-aware input processing for Kazakh programming QA. In this comparison, FEMSeg_kaz is used for preprocessing, and MAPP is used for normalization after segmentation. The purpose is to evaluate how these established components behave in a fixed low-resource educational QA setting, rather than to introduce a new neural architecture.

To isolate the effects of input preprocessing and domain-specific fine-tuning, we evaluated five distinct configurations:

- Base-only, which used the original, unprocessed question text as the zeroshot baseline.
- Base + fine-tuning, which used the original question text but applied domain-specific fine-tuning. This configuration was included to isolate the effect of fine-tuning without morphology-aware preprocessing.
- FEMSeg_kaz, which applied morphological segmentation to decompose Kazakh words into constituent morphemes.
- FEMSeg_kaz + MAPP, which combined segmentation with post-segmentation normalization to reduce representation noise and unify morphological variants.

- Fine-tuned FEMSeg_kaz + MAPP, which extended the FEMSeg_kaz + MAPP configuration with domain-specific fine-tuning.

The main methodological point is that the comparison was kept under the same conditions. Other variables were held constant, so the observed differences could be more directly linked to the input representation and its interaction with fine-tuning. In particular, the MiniLM backbone, the cleaned QA corpus, the train/test split protocol, the hyperparameters, and the evaluation metrics were the same across all configurations. This helped reduce possible confounding factors, such as model size, differences in data splitting, or inconsistencies in the evaluation process. By keeping these conditions fixed, the study provides a more reliable assessment of the stages of linguistic processing.

Reproducible FEMSeg_kaz-MAPP Workflow

The FEMSeg_kaz pipeline changes each question into a morpheme-aware form. In this form, morpheme boundaries are marked with @@. After this step, MAPP applies fixed normalization rules to reduce noise in the segmented text. In this study, MAPP used several normalization steps. First, Unicode normalization and case normalization were applied. Second, extra spaces were removed. Third, punctuation marks such as ?, ., „ and : were attached to the previous token. Fourth, segmentation markers were made consistent by removing spaces around boundary tokens. For example, de@@ gen@@ i@@ miz was changed to de@@gen@@i@@miz. The normalization also handled different forms of hyphens and apostrophes. Importantly, the morphology-aware version preserved the morpheme boundaries rather than removing them. The step-by-step MAPP post-segmentation normalization procedure is presented in Algorithm 1.

Algorithm 1. MAPP Post-Segmentation Normalization

Input: FEMSeg_kaz-segmented question q_seg
Output: normalized morphology-aware question q_mapp

1. Apply Unicode normalization to q_seg.
2. Convert ordinary natural-language tokens to lowercase.
3. Preserve programming symbols, operators, and code-related tokens where lowercasing may change their technical meaning.
4. Replace multiple whitespace characters with a single space.
5. Remove unnecessary spaces around the morpheme boundary marker "@@". Example: de@@ gen@@ i@@ miz → de@@gen@@i@@miz
6. Attach punctuation marks such as ?, ., „ :, and ; to the preceding token. Example: ne ? → ne?
7. Normalize hyphen and apostrophe variants to a consistent representation.
8. Preserve morpheme boundary markers in the encoder input representation.
9. Return the normalized morphology-aware question q_mapp.

The purpose was not to restore the text to a completely clean original form. Instead, the goal was to reduce unnecessary representation noise while still preserving useful morphological information.

A simplified example is shown below. Raw question: "Python дегеніміз не?" FEMSeg_kaz view: "Python де@@ ген@@ і@@ міз не ?" MAPP view: "python де@@ген@@і@@міз не?". In this example, FEMSeg_kaz shows the morpheme boundaries in the question. MAPP then reduces unnecessary spacing and spelling variations. Without this normalization, questions with the same meaning could be represented in more fragmented ways.

3.5. Fine-Tuning and Evaluation Protocol

Fine-tuning was carried out in a shared bi-encoder setup using the SentenceTransformer library. Each training example consisted of one cleaned question–answer pair. The question and its corresponding answer were encoded separately, and the model was trained to bring matched question–answer pairs closer in the embedding space. The models were trained with MultipleNegativesRankingLoss. In this objective, the correct paired answer serves as the positive target, while the other answers in the same mini-batch serve as in-batch negative examples.

All fine-tuned configurations used the same training setup. The models were trained for 5 epochs with a batch size of 32, a learning rate of 2×10^{-5}, and a warm-up fraction of 0.10.

The dataset was split into 90% for training and 10% for testing. The same deterministic split protocol was applied to all evaluated models and input representations to ensure a fair comparison.

To keep more data for training in this low-resource setting, no separate validation set was used. This also helped keep the setup the same across all configurations. Since there was no validation set, the models were trained for a fixed 5 epochs instead of using early stopping. This choice may increase the risk of overfitting. However, it ensures that all models are compared under the same training time conditions.

The same hyperparameters were used in all experiments. The study did not include a full search over learning rate, batch size, number of epochs, or alternative loss functions. For this reason, the reported results show how each model performed under the same training conditions. They should not be read as the highest possible performance that each architecture could achieve. The purpose of this design was not to tune each model separately. Instead, it was to compare domain-specific fine-tuning and morphology-aware input processing under stable and reproducible conditions. All final evaluations were performed on the same held-out test split. This helped provide a fair and transparent comparison across all experimental variants.

Two levels of evaluation were used in the study. The cross-model benchmark reports four main rank-1 metrics: Exact@1, TokenF1@1, Semantic@1, and BERTScoreF1@1. The controlled MiniLM comparison uses a larger set of metrics because it is the main experiment for evaluating the morphology-aware pipeline. In addition to the rank-1 metrics, it also reports Top-3 ExactHit, MeanCos@1, Recall@3, and MRR@3.

The evaluation was based on bi-encoder similarity scoring over a fixed set of candidate answers. Exact@1 was treated as a strict metric, counting as correct only when the top-1 selected answer matched the gold answer exactly. In an educational QA dataset, the same correct answer can often be written in different ways. For this reason, Exact@1 should be understood as a lower bound on exact identity. It should not be used alone to judge the system's practical usefulness. To give a fuller picture, the controlled MiniLM comparison also reports Top-3 ExactHit, Recall@3, and MRR@3. These metrics show whether semantically acceptable answers appear among the highest-ranked candidate answers, not only as the top-1 prediction.

To make the performance results easier to interpret, the revised manuscript reports interval-based interpretations together with point estimates. For the 200-item human audit, 95% confidence intervals were calculated using Wilson intervals and nonparametric bootstrap resampling. For the automatic metrics on the held-out test split, 95% bootstrap confidence intervals were also calculated for every reported metric. These intervals helped demonstrate the stability of the observed results and reduced the risk of overinterpreting small numerical differences [27].

To strengthen the statistical comparison between configurations, paired significance testing was also added in the revised analysis. Since the compared systems were evaluated

on the same held-out test items within each valid comparison, paired tests were used rather than independent-sample tests. For binary metrics, including Exact@1, Top-3 ExactHit, Semantic@1, and Recall@3, McNemar's exact test was used to compare paired per-question success and failure outcomes. For continuous or score-based metrics, including TokenF1@1, MeanCos@1, MRR@3, and BERTScoreF1@1, paired bootstrap resampling was used. In each bootstrap iteration, test items were resampled with replacement, and the mean difference between the two compared configurations was recalculated. The resulting paired tests were used to distinguish statistically supported differences from small numerical trends.

3.6. Formal Evaluation Procedure and Metric Definitions

Let $D = \{(q_i, a_i)\}\ N_{i=1}$ denote the held-out test set, where q_i is a Kazakh programming question and a_i is its gold instructional answer. Let $A = \{a_1, a_2, \ldots, a_N\}$ denote the fixed candidate answer set used during evaluation. For each experimental configuration, the same encoder E(.), candidate answer set, data split, and scoring procedure were used.

For each question q_i, the system computes the cosine similarity between the encoded question and each encoded candidate answer.

Equation (1). Cosine similarity scoring

$$s_{ij} = \cos(E(q_i), E(a_j)) = [E(q_i) \cdot E(a_j)] / [||E(q_i)|| \cdot ||E(a_j)||] \quad (1)$$

where s_{ij} is the similarity score between question q_i and candidate answer a_j.

The top-ranked answer is selected as the candidate answer with the highest cosine similarity score.

Equation (2). Top-1 answer selection

$$\hat{a}_i = a_{\hat{j}_i}, \text{ where } \hat{j}_i = \operatorname{argmax}_{1 \le j \le N} s_{ij}, \quad (2)$$

Here, $\hat{a}_i$ denotes the answer selected by the system for question q_i, and $\hat{j}_i$ is the index of the candidate answer with the highest similarity score. The ranked list of candidate answers is obtained by sorting all candidate answers in descending order of s_{ij}.

Exact@1. Exact@1 measures whether the top-ranked answer is exactly the gold answer.

Equation (3). Exact@1

$$\text{Exact@1} = (1/N) \times \Sigma_{i=1}^{N} I(\hat{j}_i = i) \quad (3)$$

where I(.) is an indicator function. It returns 1 if the condition is true and 0 otherwise. This is a strict identity-based metric. It does not give credit to semantically correct paraphrases if they are not the same gold answer.

Top-3 ExactHit. Top-3 ExactHit measures whether the gold answer appears among the three highest-ranked candidate answers.

Equation (4). Top-3 ExactHit

$$\text{Top-3 ExactHit} = (1/N) \times \Sigma_{i=1}^{N} I(i \in \text{Top3}(q_i)) \quad (4)$$

where $\text{Top3}(q_i)$ denotes the set of indices of the three candidate answers with the highest cosine similarity scores for question q_i.

TokenF1@1. TokenF1@1 is computed between the predicted top-1 answer $\hat{a}_i$ and the gold answer a_i. Before computing TokenF1@1, answers are normalized using whitespace normalization and punctuation cleanup. For morphology-aware configurations, morpheme boundary markers such as @@ are used only in the encoder input representation and are removed before token-level answer evaluation. This prevents the metric from rewarding segmentation artifacts.

Let $T(\hat{a}_i)$ denote the set of tokens in the predicted answer and $T(a_i)$ denote the set of tokens in the gold answer.

Equation (5). Token-level precision

$$\text{Precision}_i = |T(\hat{a}_i) \cap T(a_i)| / |T(\hat{a}_i)| \quad (5)$$

Equation (6). Token-level recall

$$\text{Recall}_i = |T(\hat{a}_i) \cap T(a_i)| / |T(a_i)| \quad (6)$$

Equation (7). Token-level F1

$$\text{TokenF1}_i = [2 \times \text{Precision}_i \times \text{Recall}_i]/[\text{Precision}_i + \text{Recall}_i] \quad (7)$$

If both precision and recall are zero, TokenF1_i is set to zero.

The final TokenF1@1 score is calculated as the mean TokenF1 value across all test questions.

Equation (8). TokenF1@1

$$\text{TokenF1@1} = (1/N) \times \Sigma_{i=1}{}^{N} \text{TokenF1}_i \quad (8)$$

MeanCos@1. MeanCos@1 reports the average cosine similarity between each question and its top-ranked selected answer.

Equation (9). MeanCos@1

$$\text{MeanCos@1} = (1/N) \times \Sigma_{i=1}{}^{N} \max_j s_{ij} \quad (9)$$

This metric reflects the average question–answer similarity score in the model embedding space.

Semantic@1. Semantic@1 evaluates whether the top-ranked predicted answer is semantically close to the gold answer. For this metric, the predicted answer $\hat{a}_i$ and the gold answer a_i are encoded in the answer-embedding space using the same evaluation encoder. Their cosine similarity is denoted as ans_cos_i.

Equation (10). Answer-level semantic similarity

$$\text{ans_cos}_i = \cos(E(a_i), E(\hat{a}_i)) \quad (10)$$

A prediction is counted as semantically acceptable if $\text{ans_cos}_i \geq 0.85$. The threshold of 0.85 was used as a fixed decision boundary for all configurations.

Equation (11). Semantic@1

$$\text{Semantic@1} = (1/N) \times \Sigma_{i=1}{}^{N} I(\text{ans_cos}_i \geq 0.85) \quad (11)$$

Recall@3. Recall@3 measures whether at least one of the top three selected candidate answers is semantically acceptable with respect to the gold answer.

Equation (12). Recall@3

$$\text{Recall@3} = (1/N) \times \Sigma_{i=1}{}^{N} I(\exists j \in \text{Top3}(q_i): \cos(E(a_i), E(a_j)) \geq 0.85) \quad (12)$$

This metric checks whether a semantically acceptable answer appears within the top three ranked answers.

MRR@3. MRR@3 measures the reciprocal rank of the first semantically acceptable answer within the top three candidates.

Equation (13). MRR@3

$$\text{MRR@3} = (1/N) \times \Sigma_{i=1}{}^{N} (1/r_i) \quad (13)$$

where r_i is the rank position of the first candidate answer in the top three list that satisfies the condition $\cos(E(a_i), E(a_j)) \geq 0.85$. If no candidate answer in the top three satisfies this condition, the contribution for that question is set to zero.

BERTScoreF1@1. BERTScoreF1@1 is computed between the selected top-1 answer and the gold answer. It is used as a contextual-overlap metric that captures semantic similarity beyond exact lexical overlap. The final value is reported as the mean BERTScore F1 score across the held-out test set.

This formalization ensures that all configurations are evaluated using the same answer selection procedure, candidate answer set, semantic threshold, and metric definitions.

4. Results

4.1. Cross-Model Results Before and After Fine-Tuning

The baseline models were first evaluated on the held-out test set. The results are shown in Table 6. Exact@1 was low for all three encoders and is therefore interpreted together with complementary answer quality metrics.

Table 6. Baseline model performance.

Model	Exact@1	TokenF1@1	Semantic@1 (ans_cos $\geq$ 0.85)	BERTScoreF1@1
MiniLM-L12-v2	0.013693	0.452576	0.346497	0.827941
distiluse-base-multilingual-cased-v1	0.012701	0.448055	0.238341	0.827613
bert-base-multilingual-cased	0.011907	0.440250	0.757293	0.829191

After fine-tuning, TokenF1@1 and BERTScoreF1@1 improved for all three encoder models. At the same time, Exact@1 still remained low, while Semantic@1 varied across the models. Examples before and after fine-tuning are presented in Table 7. The full post-fine-tuning results are shown in Table 8. Overall, the results suggest that fine-tuning helped the models better adapt to the programming QA domain. Still, the improvement was not equally great across all metrics. For the MiniLM configuration used in the controlled analysis, several top-k metrics also improved after fine-tuning. Top-3 ExactHit increased from 0.017265 to 0.020441. MRR@3 increased from 0.401237 to 0.413541. BERTScoreF1@1 increased from 0.827960 to 0.849063. By contrast, Recall@3 changed only slightly, from 0.471324 to 0.473110. The 95% bootstrap intervals show a similar pattern for BERTScoreF1@1. Its interval moved upward from [0.825555, 0.830420] to [0.846815, 0.851292]. For Exact@1, the intervals still overlap across the evaluated settings. Therefore, the increase in Exact@1 should be interpreted carefully. It is more appropriate to describe it as an observed trend, not as a formally confirmed improvement.

Table 7. Examples before and after fine-tuning.

Before Fine-Tuning	After Fine-Tuning
Question: What kind of values does the tuple type store in Python? Answer: It stores integers (e.g., 1, −5, 100).	Question: What kind of values does the tuple type store in Python? Answer: An immutable ordered collection (e.g., (1, 2, 3)).
Question: Can the print() function in Python output multiple values? Answer: Used to display information on the screen.	Question: Can the print() function in Python output multiple values? Answer: Yes, values can be output separated by a space or a comma (,).

Table 8. Model performance after fine-tuning.

Model	Exact@1	TokenF1@1	Semantic@1 (ans_cos $\geq$ 0.85)	BERTScoreF1@1
MiniLM-L12-v2	0.015479	0.521716	0.378051	0.849194
distiluse-base-multilingual-cased-v1	0.016472	0.519180	0.360786	0.848579
bert-base-multilingual-cased	0.015479	0.519903	0.415162	0.849739

4.2. Controlled Morphology-Aware MiniLM Comparison

Table 9 presents the controlled MiniLM comparison, the main methodological part of this study. Five configurations are evaluated: zeroshot base, base + fine-tuning, FEMSeg_kaz, FEMSeg_kaz + MAPP, and fine-tuned FEMSeg_kaz + MAPP. The base + fine-tuning condition was included as a separate configuration because it isolates the effect of domain-specific fine-tuning without morphology-aware preprocessing. This makes it possible to compare the effect of fine-tuning alone, morphology-aware preprocessing alone, and their combination under the same MiniLM backbone, data split, candidate answer set, training setup, and evaluation protocol. To ensure a fair comparison, the controlled MiniLM experiment used the same model backbone, cleaned QA corpus, data-splitting procedure, candidate answer set, training setup, evaluation protocol, and metric set for all configurations. As a result, the differences mainly stem from morphology-aware input processing, domain-specific fine-tuning, and their interaction. Along with strict top-1 semantic-matching metrics, Table 9 also reports top-k candidate-ranking indicators. These metrics provide a clearer view of how the system may behave in practice.

Table 9. Controlled MiniLM comparison across base, fine-tuning, and morphology-aware input configurations.

Metric	Zeroshot (Base)	Base + Fine-Tuning	Preprocessing (FEMSeg_kaz)	Post-Segmentation Normalization (FEMSeg_kaz + MAPP)	Fine-Tuned Post-Segmentation Normalization (FEMSeg_kaz + MAPP)
Exact@1	0.013693	0.015479	0.012518	0.015115	0.017104
Top-3 ExactHit	0.017265	0.020441	0.016417	0.020485	0.023667
TokenF1@1	0.452488	0.521716	0.448147	0.473762	0.520228
MeanCos@1 (QSim)	0.890743	0.859425	0.895775	0.878759	0.846286
Semantic@1 (ans_cos $\geq$ 0.85)	0.346299	0.378051	0.346809	0.379475	0.355012
Recall@3 (ans_cos $\geq$ 0.85)	0.471324	0.473110	0.461112	0.508950	0.474145
MRR@3 (ans_cos $\geq$ 0.85)	0.401237	0.413541	0.396778	0.436191	0.408446
BERTScoreF1@1	0.827960	0.849194	0.827587	0.835281	0.850589

Compared with the base condition, FEMSeg_kaz alone did not show a stable practical improvement. Several metrics decreased slightly. Exact@1 dropped from 0.013693 to 0.012518, and Top-3 ExactHit decreased from 0.017265 to 0.016417. TokenF1@1 also went

down from 0.452488 to 0.448147. The same pattern was seen in the top-k metrics. Recall@3 decreased from 0.471324 to 0.461112, and MRR@3 decreased from 0.401237 to 0.396778. At the same time, Semantic@1 stayed almost the same, changing only from 0.346299 to 0.346809. MeanCos@1 increased slightly from 0.890743 to 0.895775. Overall, these results suggest that segmentation alone is not sufficient to yield a consistent answer-level benefit in the shared evaluation setting.

In contrast, the FEMSeg_kaz + MAPP configuration showed the clearest morphology-only improvement over the base MiniLM condition. Exact@1 increased to 0.015115 (+10.39%), and Top-3 ExactHit increased to 0.020485 (+18.64%). TokenF1@1 also improved to 0.473762 (+4.70%). The same positive pattern appeared in the semantic and top-k ranking metrics. Semantic@1 increased to 0.379475 (+9.58%), Recall@3 to 0.508950 (+7.98%), and MRR@3 to 0.436191 (+8.71%). BERTScoreF1@1 also rose to 0.835281 (+0.89%). At the same time, MeanCos@1 (QSim) decreased slightly to 0.878759. This is the main practical result of the controlled comparison. The morphology-aware pipeline becomes useful not only through segmentation itself. It becomes more effective when the segmented representation is stabilized through deterministic MAPP normalization.

When fine-tuning was applied to FEMSeg_kaz + MAPP, the combined configuration achieved the highest values for several metrics: Exact@1 = 0.017104, Top-3 ExactHit = 0.023667, TokenF1@1 = 0.520228, and BERTScoreF1@1 = 0.850589. However, it did not outperform all other configurations on every metric. The non-fine-tuned FEMSeg_kaz + MAPP configuration achieved higher values on the threshold-based semantic ranking metrics: Semantic@1 = 0.379475, Recall@3 = 0.508950, and MRR@3 = 0.436191. In comparison, the fine-tuned FEMSeg_kaz + MAPP configuration achieved Semantic@1 = 0.355012, Recall@3 = 0.474145, and MRR@3 = 0.408446.

This pattern shows that morphology-aware normalization and domain-specific fine-tuning affect different aspects of answer selection. Fine-tuning on top of FEMSeg_kaz + MAPP improved strict top-1 identity matching, top-3 exact hit rate, token-level overlap, and contextual overlap. By contrast, FEMSeg_kaz + MAPP without fine-tuning performed better on the fixed-threshold semantic ranking metrics. Therefore, the results should not be interpreted as uniform superiority of the combined configuration across all metrics. A more precise interpretation is that FEMSeg_kaz + MAPP and fine-tuning have complementary, metric-dependent effects.

The bootstrap intervals support this interpretation in general, but the results still need to be interpreted carefully. For BERTScoreF1@1, the 95% bootstrap interval increased from [0.825555, 0.830420] in the base condition to [0.832857, 0.837559] with FEMSeg_kaz + MAPP. After fine-tuning, it increased further to [0.848470, 0.852694]. A similar pattern was seen for Recall@3. Its interval changed from [0.457630, 0.485414] in the base condition to [0.495227, 0.522474] with FEMSeg_kaz + MAPP. However, the Exact@1 intervals overlap more across the evaluated configurations. For this reason, the improvements in top-1 answer selection should be stated carefully and interpreted together with the paired significance tests reported in Table 10.

Another important point concerns MeanCos@1 (QSim). This value decreases while answer-level performance improves. This suggests that higher raw question-embedding similarity does not always mean better answer-level quality.

To further examine whether the observed differences were statistically supported, paired significance tests were conducted for configurations evaluated on matched held-out items. McNemar's exact test was used for binary metrics, including Exact@1, Top-3 ExactHit, Semantic@1, and Recall@3. Paired bootstrap resampling was used for continuous metrics, including TokenF1@1, MeanCos@1, MRR@3, and BERTScoreF1@1. Because the

per-item logs showed slight differences in the number of matched test items across some runs, paired tests were reported only for comparisons with matched item identifiers.

Table 10. Paired significance tests for the controlled comparisons.

Comparison	N	Metric	Mean A	Mean B	Difference B − A	Test	*p*-Value	Interpretation
Base + FT vs. Base	5039	Exact@1	0.013693	0.015082	+0.001389	McNemar	0.1671	Not significant
Base + FT vs. Base	5039	Top-3 ExactHit	0.017265	0.020837	+0.003572	McNemar	0.0014	Significant
Base + FT vs. Base	5039	TokenF1@1	0.452576	0.521514	+0.068937	Paired bootstrap	<0.0001	Significant
Base + FT vs. Base	5039	Semantic@1	0.346497	0.376067	+0.029569	McNemar	0.0001	Significant
Base + FT vs. Base	5039	Recall@3	0.471324	0.485215	+0.013892	McNemar	0.0644	Not significant
Base + FT vs. Base	5039	MRR@3	0.401369	0.425051	+0.023682	Paired bootstrap	0.0004	Significant
Base + FT vs. Base	5039	BERTScoreF1@1	0.827941	0.848893	+0.020953	Paired bootstrap	<0.0001	Significant
FEMSeg + MAPP vs. FEMSeg	5027	Exact@1	0.014522	0.015118	+0.000597	McNemar	0.6900	Not significant
FEMSeg + MAPP vs. FEMSeg	5027	Top-3 ExactHit	0.019097	0.020290	+0.001194	McNemar	0.3269	Not significant
FEMSeg + MAPP vs. FEMSeg	5027	TokenF1@1	0.452875	0.474104	+0.021229	Paired bootstrap	<0.0001	Significant
FEMSeg + MAPP vs. FEMSeg	5027	Semantic@1	0.340760	0.379749	+0.038989	McNemar	<0.0001	Significant
FEMSeg + MAPP vs. FEMSeg	5027	Recall@3	0.457330	0.509648	+0.052317	McNemar	<0.0001	Significant
FEMSeg + MAPP vs. FEMSeg	5027	MRR@3	0.391420	0.436675	+0.045256	Paired bootstrap	<0.0001	Significant
FEMSeg + MAPP vs. FEMSeg	5027	BERTScoreF1@1	0.828735	0.835531	+0.006796	Paired bootstrap	<0.0001	Significant
FT + FEMSeg + MAPP vs. FEMSeg + MAPP	5028	Exact@1	0.015115	0.017104	+0.001989	McNemar	0.0129	Significant
FT + FEMSeg + MAPP vs. FEMSeg + MAPP	5028	Top-3 ExactHit	0.020286	0.023469	+0.003182	McNemar	0.0001	Significant
FT + FEMSeg + MAPP vs. FEMSeg + MAPP	5028	TokenF1@1	0.474088	0.520100	+0.046012	Paired bootstrap	<0.0001	Significant
FT + FEMSeg + MAPP vs. FEMSeg + MAPP	5028	Semantic@1	0.379674	0.355012	−0.024662	McNemar	0.0003	Significant decrease
FT + FEMSeg + MAPP vs. FEMSeg + MAPP	5028	Recall@3	0.509547	0.473747	−0.035800	McNemar	<0.0001	Significant decrease
FT + FEMSeg + MAPP vs. FEMSeg + MAPP	5028	MRR@3	0.436588	0.408313	−0.028275	Paired bootstrap	<0.0001	Significant decrease
FT + FEMSeg + MAPP vs. FEMSeg + MAPP	5028	BERTScoreF1@1	0.835513	0.850590	+0.015077	Paired bootstrap	<0.0001	Significant

The paired tests support three main observations. First, base fine-tuning significantly improves Top-3 ExactHit, TokenF1@1, Semantic@1, MRR@3, and BERTScoreF1@1 compared with the base configuration, whereas the Exact@1 and Recall@3 differences are not statistically significant at the 0.05 level. Second, adding MAPP after FEMSeg_kaz significantly improves TokenF1@1, Semantic@1, Recall@3, MRR@3, and BERTScoreF1@1, although the Exact@1 and Top-3 ExactHit differences are not statistically significant. Third, adding fine-tuning on top of FEMSeg_kaz + MAPP significantly improves Exact@1, Top-3 ExactHit, TokenF1@1, and BERTScoreF1@1, but significantly decreases Semantic@1, Recall@3, and MRR@3. These results strengthen the revised interpretation that morphology-aware normalization and fine-tuning have complementary and metric-dependent effects rather than uniformly positive effects across all indicators.

4.3. Human Evaluation of Question-Answering Results

Manual evaluation was used as a limited human audit of answer correctness and pedagogical usefulness. It should not be interpreted as a full classroom evaluation or as direct evidence of learning gains. A 200-item audit sample was randomly selected from the held-out test set using a fixed seed. The sample was not stratified by topic or difficulty; topic- and difficulty-balanced human evaluation is left for future work. The same sample was used to compare the baseline and fine-tuned MiniLM outputs. This ensured that the two conditions were compared on the same questions. The same three experts who participated in the corpus-cleaning stage also participated in this human evaluation stage; however, the purpose of this stage was to assess model-selected answers rather than to clean the original dataset.

MiniLM was chosen for this audit because it was the main model in the controlled morphology-aware comparison and was computationally lighter during inference. For each sampled question, experts judged whether the selected answer was pedagogically acceptable for a beginner learner. The judgment considered whether the answer was technically correct, directly relevant to the question, clear enough for a novice, and free

from misleading code or explanation. In the main human audit, 200 responses were checked before and after fine-tuning. Before fine-tuning, the first expert marked 108 out of 200 responses as correct. This corresponds to 54%, as shown in Table 11. After fine-tuning, the same expert marked 124 out of 200 responses as correct. This corresponds to 62%, as shown in Table 12. On this limited audit sample, fine-tuning was associated with an 8-percentage-point increase in expert-rated correctness.

Table 11. Manually verified baseline model performance.

Total Questions	Incorrect Answers	Correct Answers	Accuracy
200	92	108	54%

Table 12. Manually verified fine-tuned model performance.

Total Questions	Incorrect Answers	Correct Answers	Accuracy
200	76	124	62%

The 95% Wilson confidence interval was 47.1–60.8% for the baseline condition. After fine-tuning, it increased to 55.1–68.4%. A nonparametric bootstrap analysis was also conducted using 10,000 resamples from the 200-item audit set. This produced very similar intervals: 47.0–61.0% for the baseline condition and 55.0–69.0% after fine-tuning. These intervals suggest an upward trend in expert-rated correctness after fine-tuning on the selected audit sample. However, the human audit should not be treated as a standalone proof of significance. It should be interpreted together with the full automatic evaluation. To check annotation consistency, the same 200-item sample was also reviewed by three experts. The combined expert judgments are summarized in Table 13.

Table 13. Expert (Cohen's kappa) evaluation results.

Mode	Expert 1: Correct (n, %)	Expert 1: Incorrect (n, %)	Expert 2: Correct (n, %)	Expert 2: Incorrect (n, %)	Expert 3: Correct (n, %)	Expert 3: Incorrect (n, %)	Total (n)
Baseline	108 (54%)	92 (46%)	110 (55%)	90 (45%)	109 (54.5%)	91 (45.5%)	200
After fine-tuning	124 (62%)	76 (38%)	126 (63%)	74 (37%)	125 (62.5%)	75 (37.5%)	200

Inter-annotator agreement was measured using Cohen's kappa. In the baseline condition, the chance agreement value was Pe = 0.5040. After fine-tuning, the final value of Pe was 0.5312. The observed agreement was Po = 0.58 for the baseline condition. After fine-tuning, it increased to Po = 0.63. Using the standard Cohen's kappa formula, the baseline result was $\kappa = 0.1532$. After fine-tuning, the value increased to $\kappa = 0.2108$. Pairwise agreement results and the average kappa values are reported in Table 14.

Table 14. Computed Cohen's kappa values ($\kappa_avg = (\kappa_{12} + \kappa_{13} + \kappa_{23})/3$).

Mode	Pair	Po (agree/N)	Pe (Chance)	κ	κ_avg	Interpretation
Baseline	E1–E2	0.58	0.5040	0.1532	0.1532	Slight agreement
	E1–E3	0.57	0.5030	0.1438	0.1532	
	E2–E3	0.585	0.5047	0.1625	0.1532	
After fine-tuning	E1–E2	0.63	0.5312	0.2108	0.2108	Fair agreement
	E1–E3	0.625	0.5306	0.2021	0.2108	
	E2–E3	0.635	0.5319	0.2193	0.2108	

Both kappa values fall within the slight-to-fair agreement range. This indicates that expert judgment in educational QA evaluation is partly subjective, especially when binary labels are used for answers that may be partially correct, incomplete, or pedagogically useful in different ways. The observed agreement increased from 0.58 in the baseline condition to 0.63 after fine-tuning, but the corresponding kappa values remained low. Therefore, the human evaluation results should be interpreted cautiously. They provide supplementary evidence that fine-tuning improved expert-rated correctness on the selected audit sample, but they do not constitute strong evidence of broad educational effectiveness or high annotation reliability.

5. Discussion

The cross-model benchmark shows that domain-specific fine-tuning is helpful but not sufficient on its own. Across the three encoder families, fine-tuning improved token-level overlap, BERTScoreF1@1, and expert-rated correctness, while Exact@1 remained low. This is expected because Exact@1 is a strict identity-based metric over a large candidate answer set. Therefore, the results should be interpreted across complementary metrics, including TokenF1@1, Semantic@1, Top-3 ExactHit, Recall@3, MRR@3, and BERTScoreF1@1, rather than through Exact@1 alone.

The low absolute Exact@1 values also define the practical scope of the present system. In a candidate set of 50,386 answers, Exact@1 is a strict identity-based metric and should be interpreted as a conservative indicator of exact answer selection rather than as a direct measure of educational usefulness. The practical relevance of the results therefore lies not in achieving high absolute Exact@1, but in showing whether morphology-aware normalization and fine-tuning produce consistent changes across complementary metrics. Improvements in TokenF1@1, BERTScoreF1@1, Top-3 ExactHit, Recall@3, MRR@3, and expert-audit correctness provide a broader view of answer quality. Nevertheless, the low Exact@1 values show that the current system should not yet be treated as a fully reliable classroom QA assistant without additional ranking improvements, answer generation support, or teacher-supervised validation.

One metric pattern needs a separate explanation. In Tables 6 and 8, the bert-base-multilingual-cased model has a relatively high Semantic@1 score before fine-tuning. After fine-tuning, this value becomes lower. This does not necessarily conflict with the improvements in TokenF1@1 and BERTScoreF1@1. Semantic@1 depends on a fixed threshold: ans_cos ≥ 0.85. As a result, even small changes in the answer-embedding space can move many examples above or below the threshold. In other words, fine-tuning may make the predicted answers lexically stronger or more useful for teaching. At the same time, some answers may shift around the fixed semantic cutoff. For this reason, threshold-based semantic metrics should not be interpreted alone. They should be considered together with broader top-k answer selection metrics. The unusually high zero-shot Semantic@1 value for bert-base-multilingual-cased is likely related to the score distribution of the BERT answer-embedding space. More specifically, many predicted and gold answer representations appear to fall near or above the fixed ans_cos ≥ 0.85 threshold. Because this pattern is not accompanied by a comparable advantage in Exact@1, TokenF1@1, or BERTScoreF1@1, we interpret it as a threshold-sensitive embedding space effect and a possible evaluation artifact rather than as evidence of substantially better zero-shot answer quality.

The controlled MiniLM comparison shows the main empirical contribution more clearly than the cross-model benchmark alone. In this experiment, the model backbone, data split protocol, answer space, training schedule, evaluation setup, and reported metrics were kept the same across all configurations.

Even under these controlled conditions, morphology-aware input processing still changed the results. However, not all variants were equally useful. FEMSeg_kaz alone slightly increased question-level cosine similarity, but it did not improve the practical answer-level results. This suggests that raw segmentation by itself is not enough. In contrast, FEMSeg_kaz + MAPP produced the clearest morphology-only gains over the base MiniLM condition. Improvements were observed in Exact@1, Top-3 ExactHit, TokenF1@1, Semantic@1, Recall@3, MRR@3, and BERTScoreF1@1. This result supports the paper's main idea. Segmentation becomes more useful when it is followed by deterministic normalization. MAPP reduces spelling and spacing variation while keeping useful morpheme boundary information.

Another important finding appears when fine-tuning is added on top of FEMSeg_kaz + MAPP. The combined configuration achieved the highest values among the controlled MiniLM variants for Exact@1, Top-3 ExactHit, TokenF1@1, and BERTScoreF1@1. This suggests that morphology-aware input processing and domain-specific fine-tuning can support each other for strict answer identity, exact top-k matching, token overlap, and contextual overlap. However, this improvement was not uniform across all metrics. The non-fine-tuned FEMSeg_kaz + MAPP configuration performed better on Semantic@1, Recall@3, and MRR@3. These metrics depend on a fixed semantic threshold and may respond differently to embedding-space shifts introduced by fine-tuning. Therefore, the revised interpretation is more cautious: fine-tuning on top of FEMSeg_kaz + MAPP improves several answer selection and contextual-overlap indicators, whereas FEMSeg_kaz + MAPP without fine-tuning remains stronger on some threshold-based semantic ranking indicators.

The bootstrap intervals suggest that these results should be interpreted carefully.

For BERTScoreF1@1 and Recall@3, the intervals show a steady upward trend for FEMSeg_kaz + MAPP compared with the base condition.

At the same time, the Exact@1 intervals overlap more across the different configurations. For this reason, small differences in automatic metrics should not be treated as confirmed improvements. A more careful interpretation divides the results into three main points. First, the confidence intervals support some improvements at the answer level. Second, results based on threshold-dependent metrics should be interpreted with caution. Third, the improvements in top-1 answer selection remain practically important, but they should be described with caution. There is also an important pattern in MeanCos@1 (QSim). The best answer-level results did not come from the highest question-level cosine similarity. This suggests that raw question-embedding similarity is not always the best indicator of better answer quality.

For Kazakh educational question answering, the main practical message is that improvement is more reliable when several established components are combined and evaluated under controlled conditions. These include expert-cleaned domain data, shared bi-encoder fine-tuning, linguistically informed preprocessing, and mixed evaluation methods. The evaluation process should include both automatic metrics and human judgment. The results do not support the simple idea that a larger encoder is sufficient on its own. Instead, they show that morphology-aware normalization can be a useful adaptation step when tested within a controlled, reproducible evaluation framework. This interpretation also clarifies the type of QA framework used in this study. The proposed system is an encoder-based semantic answer selection framework. In this framework, fine-tuning improves the alignment between Kazakh programming questions and their corresponding answers in the embedding space. Therefore, MiniLM and the other encoder models are evaluated through answer selection, semantic matching, and contextual overlap.

The educational contribution of this study should be interpreted at the pre-deployment evaluation level. The study does not claim that the proposed pipeline has already improved

student learning outcomes in a classroom setting. Rather, it shows that a Kazakh-language programming QA system can be evaluated under controlled answer selection conditions and that morphology-aware normalization can improve several automatic and expert-audit indicators. In this sense, the practical contribution is methodological and infrastructural: the study provides a curriculum-guided Kazakh programming QA corpus, a controlled evaluation protocol, morphology-aware preprocessing scripts, and evidence that such preprocessing can affect answer selection quality. These components can support future development of deployable Kazakh educational QA systems, but classroom effectiveness still requires separate validation with students, teachers, and learning platforms.

One possible concern is that the morphology-aware effect may be specific to MiniLM, because the controlled morphology-aware experiment was conducted with this model. This design was intentional. By keeping the backbone model fixed, the study could examine input processing more clearly and separate the effect of morphology-aware normalization from differences in model size, architecture, and training behavior.

The cross-model benchmark in Table 8 provides additional context for the effect of domain-specific fine-tuning across multilingual encoders. After fine-tuning, MiniLM-L12-v2 achieved Exact@1 = 0.015479 and BERTScoreF1@1 = 0.849194, distiluse-base-multilingual-cased-v1 achieved Exact@1 = 0.016472 and BERTScoreF1@1 = 0.848579, and bert-base-multilingual-cased achieved Exact@1 = 0.015479 and BERTScoreF1@1 = 0.849739. These values are consistent with the results reported in Table 8.

Therefore, the controlled MiniLM experiment should be interpreted as the main experiment for isolating the effect of morphology-aware input processing. The cross-model benchmark supports the general usefulness of domain-specific fine-tuning, but it should not be used to claim that the morphology-aware effect has the same magnitude across all encoder families. Future work should apply the same leak-controlled morphology-aware protocol to additional encoder models.

Several limitations remain in this study. First, the controlled morphology-aware analysis was conducted primarily with MiniLM. Therefore, the gains from FEMSeg_kaz + MAPP should still be tested with other encoder families under the same leak-controlled protocol.

Second, although the revised analysis includes paired significance tests, the results should still be interpreted cautiously. McNemar's exact test was used for binary metrics, and paired bootstrap resampling was used for continuous metrics. These tests helped distinguish statistically supported differences from small numerical trends. However, several observed improvements remain small in magnitude, especially for strict Exact@1. Therefore, the results should not be interpreted as evidence that one configuration is uniformly superior across all metrics.

Third, the dataset was generated with LLM assistance, but the generation process was curriculum-guided and expert-validated. The topic structure was prepared according to the school programming curriculum and the teacher's long-term instructional plan, and the generated items were checked by three experts for technical correctness, relevance, clarity, code validity, and pedagogical usefulness. Therefore, the corpus should not be understood as an arbitrary GPT-generated dataset. It is better understood as a controlled curriculum-aligned educational QA benchmark. However, it was not collected from authentic classroom logs, LMS records, or real-time student interactions. Consequently, the external validity of the reported results is limited: the findings demonstrate performance on a curriculum-guided benchmark, not on authentic student question logs. Future work should evaluate the proposed approach on real student questions collected from classroom practice, LMS records, or programming tutor interaction logs.

Fourth, another limitation concerns the human evaluation. The manual audit included 200 model-selected responses, which is a small subset of the full corpus and the held-out test

set. The sample was useful for checking whether automatic improvements corresponded to some increase in expert-rated answer correctness, but it was not large enough to support strong claims about practical educational effectiveness. In addition, the Cohen's kappa values indicated only slight-to-fair agreement among the experts. This suggests that judging pedagogical correctness is partly subjective and that future work should use larger audit samples, clearer multi-level rating rubrics, topic- and difficulty-balanced sampling, and classroom-based evaluation with students.

Fifth, the study does not include a deployed educational system, classroom trial, LMS integration, or student interaction study. Therefore, the results should not be interpreted as direct evidence of learning gains, student engagement, pedagogical effectiveness, or educational usability. The evaluation remains primarily intrinsic and benchmark-based, supported by a limited expert audit. Future work should implement the QA pipeline in an actual learning environment and evaluate it with students and teachers. Such evaluation should measure learning gains, usability, engagement, error recovery, student trust, and teacher-perceived usefulness.

Sixth, another limitation concerns the use of embedding-based similarity during dataset filtering. The preliminary automatic screening used the pretrained MiniLM encoder to flag potentially weakly aligned question–answer pairs. Because MiniLM was also used in the main controlled morphology-aware experiment, this may introduce a limited similarity space bias in the absolute interpretation of the results. However, the filtering step was applied before task-specific fine-tuning, was used only as a preliminary quality control signal and did not replace expert validation. Flagged items were reviewed by the three experts using predefined acceptance criteria. In addition, all experimental configurations were evaluated on the same expert-cleaned dataset, with the same data split, candidate answer set, model backbone, and evaluation protocol. Therefore, the within-dataset comparisons remain informative for assessing the relative effect of morphology-aware input processing under fixed experimental conditions. Future work should repeat the filtering stage with independent encoders and evaluate manually sampled unfiltered subsets to test whether the observed trends remain stable when dataset filtering and model evaluation use different representation spaces.

Finally, the study did not use a separate validation split or systematic hyperparameter optimization. The same learning rate, batch size, epoch number, and loss function were kept fixed across the evaluated configurations to support a controlled comparison rather than architecture-specific optimization. For this reason, the study should mainly be understood as a controlled comparative analysis rather than as a fully optimized model selection study. Future work should introduce a dedicated validation split, conduct systematic hyperparameter search, and evaluate the pipeline on more complex multi-step programming questions. Future work should apply the same protocol to more encoder models, release paired per-item prediction files to support reproducibility, and test the approach on more complex programming questions that require multi-step reasoning.

6. Conclusions

This study evaluated multilingual question-answering models for teaching programming in Kazakh. The main focus was to examine whether morphology-aware input processing can improve answer quality beyond standard domain-specific fine-tuning. The results show that morphology-aware normalization produces measurable improvements under controlled conditions. They also show that combining normalization with fine-tuning improves several answer selection and contextual-overlap metrics, although the improvement is not uniform across all reported indicators.

The controlled MiniLM experiments show that morphology-aware processing has its own effect under fixed experimental conditions. FEMSeg_kaz alone did not produce stable improvements across the metrics. However, FEMSeg_kaz + MAPP produced clearer improvements than segmentation alone, indicating that segmentation becomes more useful when it is stabilized through deterministic post-segmentation normalization.

When domain-specific fine-tuning was applied on top of FEMSeg_kaz + MAPP, the combined configuration achieved the highest values for Exact@1, Top-3 ExactHit, TokenF1@1, and BERTScoreF1@1. However, the non-fine-tuned FEMSeg_kaz + MAPP configuration remained stronger on the threshold-based semantic ranking metrics, including Semantic@1, Recall@3, and MRR@3. Therefore, the results should not be interpreted as uniform superiority of the combined configuration across all metrics. Instead, they indicate that morphology-aware normalization and fine-tuning have complementary and metric-dependent effects.

At the same time, the findings show that no single evaluation metric can fully capture answer quality in low-resource educational settings. Because of this, the results should be interpreted together across several complementary metrics. Overall, the study evaluates a morphology-aware adaptation strategy that combines FEMSeg_kaz preprocessing, MAPP normalization, and domain-specific fine-tuning for Kazakh-language educational QA systems.

The results address all three research questions. For RQ1, the findings show that domain-specific fine-tuning improves token-level and contextual-similarity metrics. For RQ2, the results indicate that morphology-aware processing is more useful when segmentation is stabilized through MAPP normalization. For RQ3, the results show that the combination of FEMSeg_kaz, MAPP, and fine-tuning improves Exact@1, Top-3 ExactHit, TokenF1@1, and BERTScoreF1@1, but not Semantic@1, Recall@3, or MRR@3. This supports a more cautious interpretation: the combined configuration improves several answer selection and contextual-overlap indicators, while the non-fine-tuned FEMSeg_kaz + MAPP configuration remains stronger on some fixed-threshold semantic ranking indicators. Because the experiments used a fixed 5-epoch training schedule without a separate validation split or early stopping, the results provide controlled comparative evidence, not proof of an optimal training configuration or fully optimized model performance.

Future work should evaluate additional model architectures, test the pipeline on more complex multi-step programming questions, and expand the diversity of the educational QA dataset. More importantly, the next stage should move beyond intrinsic answer selection evaluation by integrating the system into a real or simulated learning environment. Classroom studies, LMS-based trials, and student interaction analyses are needed to measure learning gains, engagement, usability, and teacher-perceived pedagogical usefulness.

Author Contributions: Conceptualization, U.T.; methodology, U.T. and B.H.; software, U.T. and B.R.; experiments, U.T. and B.R.; validation, U.T. and B.R.; formal analysis, U.T., B.H. and B.R.; investigation, U.T., B.H. and B.R.; resources, U.T. and B.R.; data curation, U.T. and B.R.; writing—original draft preparation, U.T. and B.R.; writing—review and editing, U.T., B.H. and B.R.; visualization, B.R.; supervision, U.T.; project administration, U.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the Ministry of Science and Higher Education of the Republic of Kazakhstan within the framework of the scientific project "Study of neural models for the formation of transcripts of speech and minutes of meetings in Turkic languages" (Grant No. AP23487816).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The software release version v1.0.0, morphology-aware preprocessing scripts, and reproducibility instructions used in this study are publicly available at the project GitHub v1.0.0 repository: https://github.com/rsbekarys-create/TEACHING-PROGRAMMING- (accessed on 21 May 2026). Due to storage and licensing constraints, the full cleaned QA corpus is not redistributed directly in the repository; instead, the repository provides data format specifications, representative samples, and instructions for reproducing the experiments and accessing the study materials.

Acknowledgments: Use of generative AI and AI-assisted technologies in the writing process. During the preparation of this work, the authors used ChatGPT, GPT-5.5 Thinking (OpenAI), and Grammarly PRO to improve the manuscript's readability and make it more academically sound and coherent. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Soares, M.A.C.; Parreiras, F.S. A literature review on question answering techniques, paradigms and systems. *J. King Saud Univ.-Comput. Inf. Sci.* **2020**, *32*, 635–646. [CrossRef]
2. Joshi, P.; Santy, S.; Budhiraja, A.; Bali, K.; Choudhury, M. The State and Fate of Linguistic Diversity and Inclusion in the NLP World. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 6282–6293. [CrossRef]
3. Tukeyev, U.; Rysbek, B. Morphology-Aware Segmentation and Tokenization for Turkic Languages: A CSE-Guided Framework (The Kazakh Case). *Information* **2026**, *17*, 128. [CrossRef]
4. Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, 2–7 June 2019*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; pp. 4171–4186. [CrossRef]
5. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, Hong Kong, China, 3–7 November 2019*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; pp. 3982–3992. [CrossRef]
6. Clark, J.H.; Choi, E.; Collins, M.; Garrette, D.; Kwiatkowski, T.; Nikolaev, V.; Palomaki, J. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Trans. Assoc. Comput. Linguist.* **2020**, *8*, 454–470. [CrossRef]
7. Lewis, P.; Oguz, B.; Rinott, R.; Riedel, S.; Schwenk, H. MLQA: Evaluating Cross-lingual Extractive Question Answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 7315–7330. [CrossRef]
8. Longpre, S.; Lu, Y.; Daiber, J. MKQA: A Linguistically Diverse Benchmark for Multilingual Open Domain Question Answering. *Trans. Assoc. Comput. Linguist.* **2021**, *9*, 1389–1406. [CrossRef]
9. Gururangan, S.; Marasović, A.; Swayamdipta, S.; Lo, K.; Beltagy, I.; Downey, D.; Smith, N.A. Don't Stop Pretraining: Adapt Language Models to Domains and Tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 8342–8360. [CrossRef]
10. Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W.-t. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 6769–6781. [CrossRef]
11. Zhang, X.; Thakur, N.; Ogundepo, O.; Kamalloo, E.; Alfonso-Hermelo, D.; Li, X.; Liu, Q.; Rezagholizadeh, M.; Lin, J. MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages. *Trans. Assoc. Comput. Linguist.* **2023**, *11*, 1114–1131. [CrossRef]
12. Zhang, X.; Ma, X.; Shi, P.; Lin, J. Mr. TyDi: A Multi-lingual Benchmark for Dense Retrieval. In *Proceedings of the 1st Workshop on Multilingual Representation Learning, Punta Cana, Dominican Republic, 11 November 2021*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2021; pp. 127–137. [CrossRef]
13. Zhang, T.; Kishore, V.; Wu, F.; Weinberger, K.Q.; Artzi, Y. BERTScore: Evaluating Text Generation with BERT. In Proceedings of the International Conference on Learning Representations, Addis Ababa, Ethiopia, 26–30 April 2020.
14. Farea, A.; Yang, Z.; Duong, K.; Perera, N.; Emmert-Streib, F. Evaluation of Question Answering Systems: Complexity of Judging a Natural Language. *ACM Comput. Surv.* **2025**, *58*, 1–43. [CrossRef]
15. van der Lee, C.; Gatt, A.; van Miltenburg, E.; Krahmer, E. Human evaluation of automatically generated text: Current trends and best practice guidelines. *Comput. Speech Lang.* **2021**, *67*, 101151. [CrossRef]

16. Qiu, W.; Su, C.L.; Jamil, N.B.; Thway, M.; Ng, S.S.H.; Zhang, L.; Lim, F.S.; Lai, J.W. A Systematic Approach to Evaluate the Use of Chatbots in Educational Contexts: Learning Gains, Engagements and Perceptions. *Computers* **2025**, *14*, 270. [CrossRef]
17. Nainia, A.; Vignes-Lebbe, R.; Mousannif, H.; Zahir, J. Beyond BLEU: Ethical Risks of Misleading Evaluation in Domain-Specific QA with LLMs. In *Proceedings of the First Workshop on Comparative Performance Evaluation: From Rules to Language Models, Varna, Bulgaria, 11 September 2025*; INCOMA Ltd.: Shoumen, Bulgaria, 2025; pp. 77–86.
18. Yeshpanov, R.; Efimov, P.; Boytsov, L.; Shalkarbayuli, A.; Braslavski, P. KazQAD: Kazakh Open-Domain Question Answering Dataset. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation, Torino, Italy, 20–25 May 2024*; ELRA and ICCL: Torino, Italy, 2024; pp. 9645–9656.
19. Togmanov, M.; Mukhituly, N.; Turmakhan, D.; Mansurov, J.; Goloburda, M.; Sakip, A.; Xie, Z.; Wang, Y.; Syzdykov, B.; Laiyk, N.; et al. KazMMLU: Evaluating Language Models on Kazakh, Russian, and Regional Knowledge of Kazakhstan. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, Vienna, Austria, 27 July–1 August 2025*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 14403–14416.
20. Mansurova, A.; Tleubayeva, A.; Nugumanova, A.; Shomanov, A.; Seker, S.E. A Systematic Evaluation of Large Language Models and Retrieval-Augmented Generation for the Task of Kazakh Question Answering. *Information* **2025**, *16*, 943. [CrossRef]
21. Wiemerslage, A.; Silfverberg, M.; Yang, C.; McCarthy, A.D.; Nicolai, G.; Colunga, E.; Kann, K. Morphological Processing of Low-Resource Languages: Where We Are and What's Next. In *Proceedings of the Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, 22–27 May 2022*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2022; pp. 988–1007. [CrossRef]
22. Mager, M.; Oncevay, A.; Mager, E.; Kann, K.; Vu, N.T. BPE vs. Morphological Segmentation: A Case Study on Machine Translation of Four Polysynthetic Languages. In *Proceedings of the Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, 22–27 May 2022*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2022; pp. 961–971. [CrossRef]
23. Blevins, T.; Zettlemoyer, L. Better Character Language Modeling through Morphology. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; pp. 1606–1613. [CrossRef]
24. Bostrom, K.; Durrett, G. Byte Pair Encoding is Suboptimal for Language Model Pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online, 16–20 November 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 4617–4624. [CrossRef]
25. Abudouwaili, G.; Ruzmamat, S.; Abiderexiti, K.; Wu, B.; Wumaier, A. A Benchmark for Morphological Segmentation in Uyghur and Kazakh. *Appl. Sci.* **2024**, *14*, 5369. [CrossRef]
26. Dror, R.; Baumer, G.; Shlomov, S.; Reichart, R. The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, Melbourne, Australia, 15–20 July 2018*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2018; pp. 1383–1392. [CrossRef]
27. Card, D.; Henderson, P.; Khandelwal, U.; Jia, R.; Mahowald, K.; Jurafsky, D. With Little Power Comes Great Responsibility. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 9263–9274. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.